Kaggle 3 Report

Dai Dasen

November 13, 2025

1 Introduction

The goal of this work is to perform **node classification** on a citation-style graph dataset. Each node is associated with a bag-of-words feature vector, and edges represent citation links. The task is to predict the class label of each node using only graph structure and node features. This problem is a standard benchmark for evaluating graph neural network (GNN) models.

2 Motivation

Classical methods treat each node independently, ignoring the relational inductive bias inherent in graphs. Graph Attention Networks (GAT) incorporate both feature information and graph topology while learning **adaptive attention weights**, enabling the model to focus on the most informative neighbors.

In this project, we use the more expressive **GATv2** operator, which improves upon the original GAT by allowing dynamic attention coefficients that depend on both source and target nodes.

3 Data Processing

Given the input text files containing node features, edges, and labeled nodes, we performed the following preprocessing steps:

- **Row-normalization of features**: To stabilize training on sparse BoW vectors.
- **Mapping raw node IDs to continuous indices**: Required for PyTorch Geometric.
- **Converting directed edges to undirected edges**: Following common practice for citation networks.
- **Adding self-loops**: Improves GAT stability by allowing nodes to attend to themselves.

- **Handling label imbalance**: We compute inverse-frequency class weights for the loss function.

These steps follow best practices from the original GAT paper and subsequent state-of-the-art node classification pipelines.

4 Model Architecture

We use a two-layer GATv2 architecture implemented with `GATv2Conv`. The model consists of:

- A first GATv2 layer with multi-head attention:

$$\text{GATv2Conv}(F_{\text{in}}, H, \text{heads} = 8),$$

  producing $8H$ output channels.

- ELU activation.
- Dropout applied before and between layers.
- A second GATv2 layer:

$$\text{GATv2Conv}(8H, C, \text{heads} = 1),$$

  where C is the number of classes.

- Log-Softmax output for use with NLLLoss.

The two-layer GAT structure is standard and proven effective on Cora-like datasets.

5 Training Strategy

We employ several training techniques commonly used in state-of-the-art GAT implementations:

- **Weighted NLL Loss**: Mitigates class imbalance in the training set.
- **Adam optimizer** with learning rate 0.005 and weight decay 5e-4.
- **Dropout = 0.6**: Helps prevent overfitting on small citation graphs.
- **EarlyStopping** with patience = 30 based on validation accuracy.
- **Model checkpointing**: Save the best model during training.

All hyperparameters are chosen based on recommendations from the original GAT and GATv2 papers.

6 Prediction and Submission

After training, we:

- Load the best model based on validation accuracy.
- Compute predictions for all nodes.
- Extract predictions for nodes in the test split.
- Map predicted class indices back to original labels.
- Save results into a CSV file for submission.

7 Conclusion

In this project, we implemented a complete GATv2-based node classification pipeline including data preprocessing, model design, training with regularization, and prediction. The combination of graph attention, row-normalized features, weighted loss, and early stopping provides a robust and modern solution for semi-supervised node classification tasks.

Future work may include experimenting with:

- Deeper GAT architectures.
- Normalized attention (e.g., AGNN).
- Residual connections or layer normalization.
- Combining GAT with graph augmentations.